

Experiment 09 – PyTest HTML Reporting

Module Name: Module 2 – Unit Test Frameworks

Experiment Number: 09

Experiment Title: PyTest HTML Reporting

Programming Language: Python

Automation Tool: Selenium WebDriver

Testing Framework: PyTest

Reporting Tool: pytest-html

Website: SauceDemo

Website URL: <https://www.saucedemo.com/>

1. Problem Statement

Manual testing of web applications can be time-consuming and repetitive, especially when multiple login scenarios need to be tested. Automated testing helps reduce manual effort and ensures consistent execution of test cases.

The objective of this experiment is to automate login testing on the SauceDemo website using Selenium WebDriver and PyTest. Three scenarios will be tested: successful login, invalid login, and locked-out user login.

The experiment also involves generating an HTML test report using the `pytest-html` plugin. Screenshots will be captured at important stages of test execution to provide visual evidence of the test results.

2. Objective

1. To understand the PyTest testing framework and its use in Selenium automation.
 2. To automate multiple login scenarios using Selenium WebDriver.
 3. To use PyTest fixtures for browser setup and teardown.
 4. To implement explicit waits for reliable element interaction.
 5. To capture screenshots during different stages of test execution.
 6. To generate an HTML test report using the `pytest-html` plugin.
 7. To verify the execution results of successful, invalid, and locked-out user login tests.
-

3. Tools, Software and Concepts Used

Tools and Software

Tool	Description
Python	Programming language used for automation.
Selenium WebDriver	Automates browser interactions.
PyTest	Framework used to organize and execute test cases.
pytest-html	Plugin used to generate HTML test reports.
Google Chrome	Browser used for automation.
Visual Studio Code	Code editor used to develop and execute the script.

Concepts Used

- **PyTest Fixtures:** Used to initialize and close the browser for each test.
- **Assertions:** Used to verify expected test results.
- **Explicit Waits:** Used to wait for web elements to become visible.
- **Screenshot Capture:** Used to save screenshots during test execution.
- **HTML Reporting:** Used to generate a detailed report of test execution.
- **Data-Driven Test Scenarios:** Multiple login scenarios are tested using separate test functions.

Installation

The required libraries can be installed using:

```
pip install selenium pytest pytest-html
```

4. Source Code / Implementation Steps

Step 1: Create the Project Structure

Create a folder named `Experiment 09 - PyTest HTML Reporting` with the following structure:

```
Experiment 09 - PyTest HTML Reporting
├── test_login.py
├── pytest.ini
├── screenshots
│   ├── 01_login_page.png
│   ├── 02_credentials_credentials.png
│   ├── 03_successful_login.png
│   ├── 04_credentials_credentials.png
│   ├── 05_invalid_login_error.png
│   ├── 06_credentials_credentials.png
│   └── 07_locked_out_error.png
├── reports
│   └── report.html
```

The `screenshots` and `reports` directories are used to store screenshots and the generated HTML report, respectively.

Step 2: Configure PyTest

Create a file named `pytest.ini` and add the following configuration:

```
[pytest]
addopts = -v
testpaths = .
```

This configuration enables verbose test output and specifies the current directory for test discovery.

Step 3: Implement the Automation Script

Create a Python file named `test_login.py` and add the following code:

```
import os
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Create folders if they do not exist
os.makedirs("screenshots", exist_ok=True)
os.makedirs("reports", exist_ok=True)

@pytest.fixture
def driver():
    browser = webdriver.Chrome()
    browser.maximize_window()
    yield browser
    browser.quit()

def login(driver, username, password, screenshot_name):
    driver.get("https://www.saucedemo.com/")
    time.sleep(2)

    # Screenshot 1: Login page
    if username == "standard_user":
        driver.save_screenshot("screenshots/01_login_page.png")

    wait = WebDriverWait(driver, 15)

    # Enter username
    wait.until(
        EC.visibility_of_element_located((By.ID, "user-name"))
    ).send_keys(username)

    # Enter password
    driver.find_element(By.ID, "password").send_keys(password)
    time.sleep(2)
```

```

# Capture entered credentials
driver.save_screenshot(
    f"screenshots/{screenshot_name}_credentials.png"
)

# Click login button
driver.find_element(By.ID, "login-button").click()
time.sleep(2)

def test_successful_login(driver):
    login(driver, "standard_user", "secret_sauce", "02")

    wait = WebDriverWait(driver, 15)
    wait.until(
        EC.visibility_of_element_located(
            (By.CLASS_NAME, "inventory_list")
        )
    )

    assert "/inventory.html" in driver.current_url

    # Screenshot 3: Successful login
    driver.save_screenshot("screenshots/03_successful_login.png")
    print("Successful login test passed.")
    time.sleep(2)

def test_invalid_login(driver):
    login(driver, "invalid_user", "wrong_password", "04")

    wait = WebDriverWait(driver, 15)
    error = wait.until(
        EC.visibility_of_element_located(
            (By.CSS_SELECTOR, "h3[data-test='error']")
        )
    )

    assert "Username and password do not match" in error.text

    # Screenshot 5: Invalid login error
    driver.save_screenshot("screenshots/05_invalid_login_error.png")
    print("Invalid login test passed.")
    time.sleep(2)

def test_locked_out_user(driver):
    login(driver, "locked_out_user", "secret_sauce", "06")

    wait = WebDriverWait(driver, 15)
    error = wait.until(
        EC.visibility_of_element_located(
            (By.CSS_SELECTOR, "h3[data-test='error']")
        )
    )

    assert "locked out" in error.text

    # Screenshot 7: Locked-out user error
    driver.save_screenshot("screenshots/07_locked_out_error.png")

```

```
print("Locked-out user test passed.")
time.sleep(2)
```

Step 4: Execute the Test Script

Open the terminal in the experiment folder and execute:

```
pytest test_login.py --html=reports/report.html --self-contained-html
```

The script performs the following operations:

1. Opens the SauceDemo website in Chrome.
2. Captures the initial login page.
3. Enters login credentials and captures screenshots.
4. Tests successful login using valid credentials.
5. Tests login with invalid credentials.
6. Tests login with a locked-out user account.
7. Captures screenshots of the resulting pages and messages.
8. Closes the browser after each test.
9. Generates an HTML report containing the test results.

A two-second delay is added after important actions to make the automation easier to observe during the demonstration. Explicit waits are also used to handle page loading and element visibility.

Step 5: Generate the HTML Report

The following command generates the HTML report:

```
pytest test_login.py --html=reports/report.html --self-contained-html
```

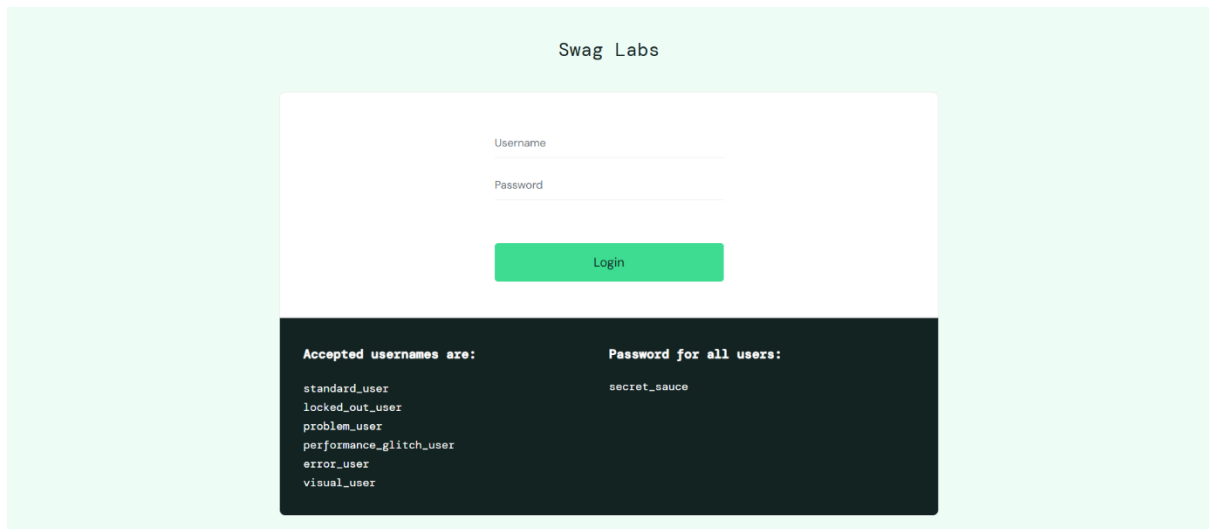
The `--html` option specifies the report location, while `--self-contained-html` generates a standalone HTML report.

The report can be opened in a web browser using:

```
start reports\report.html
```

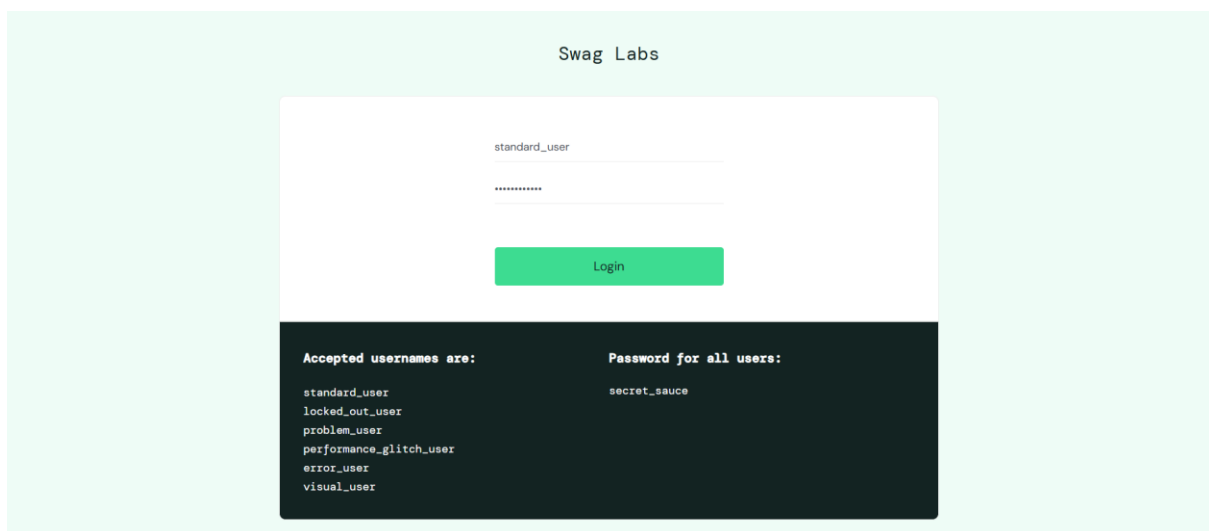
5. Output with Clearly Labelled Screenshots

Screenshot 1: SauceDemo Login Page



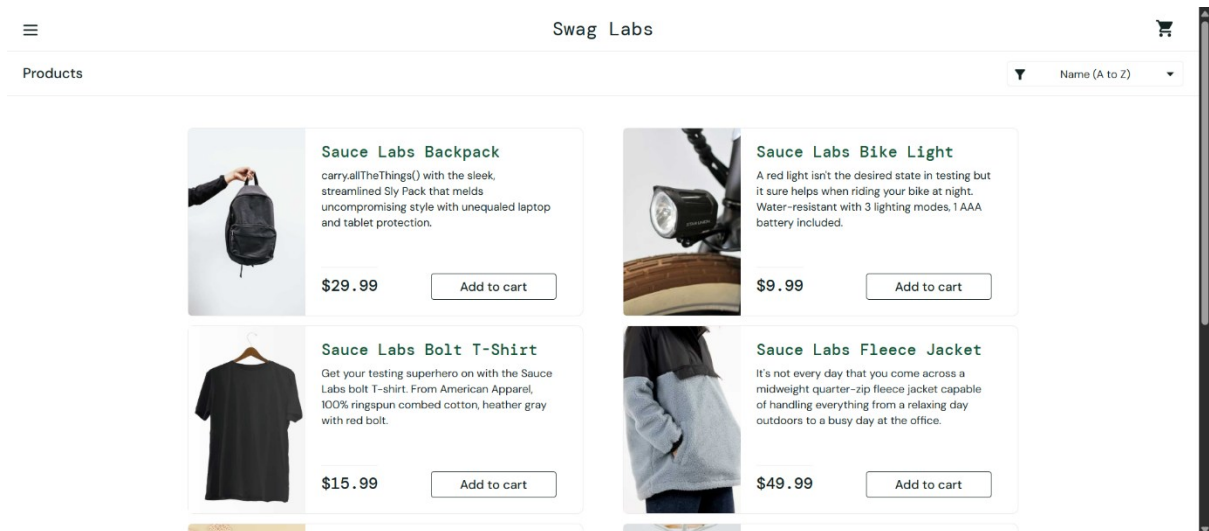
This screenshot shows the initial login page of the SauceDemo website before entering credentials.

Screenshot 2: Valid Login Credentials Entered



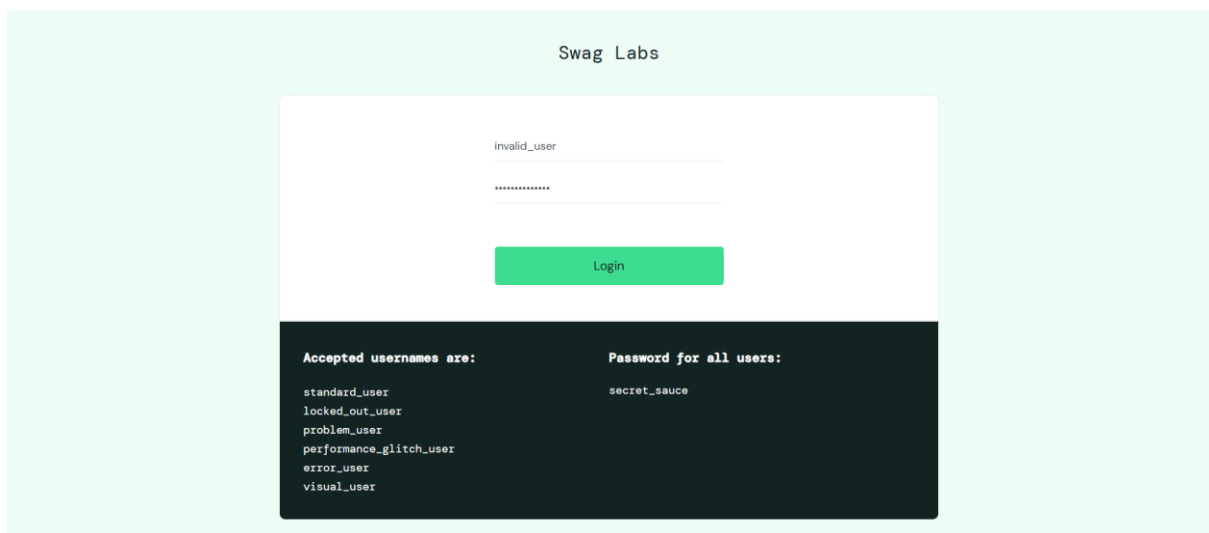
This screenshot shows the username and password entered for the successful login test.

Screenshot 3: Successful Login



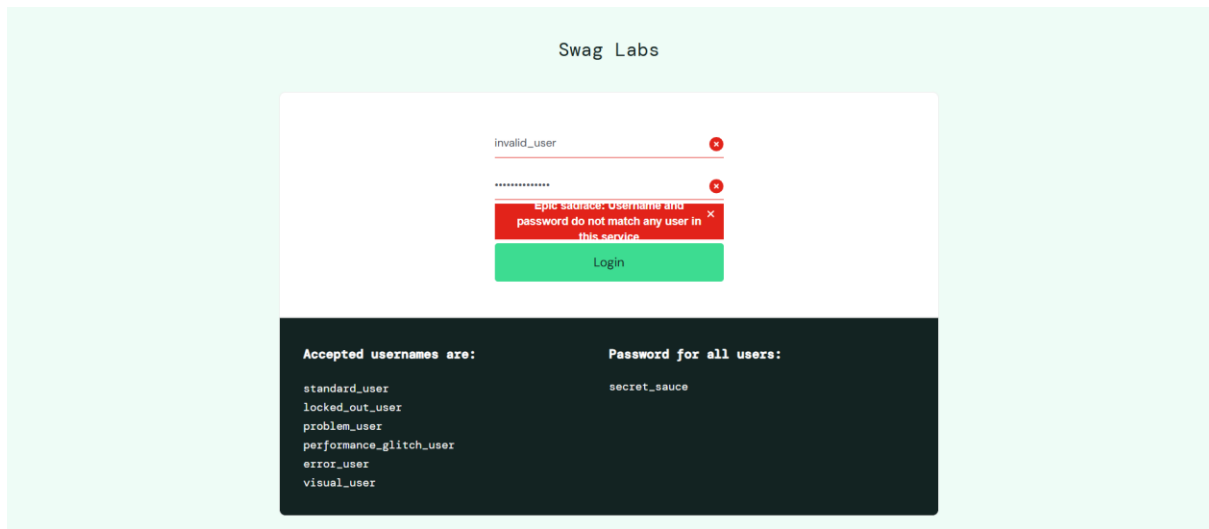
This screenshot displays the Products page after a successful login using valid credentials.

Screenshot 4: Invalid Login Credentials Entered



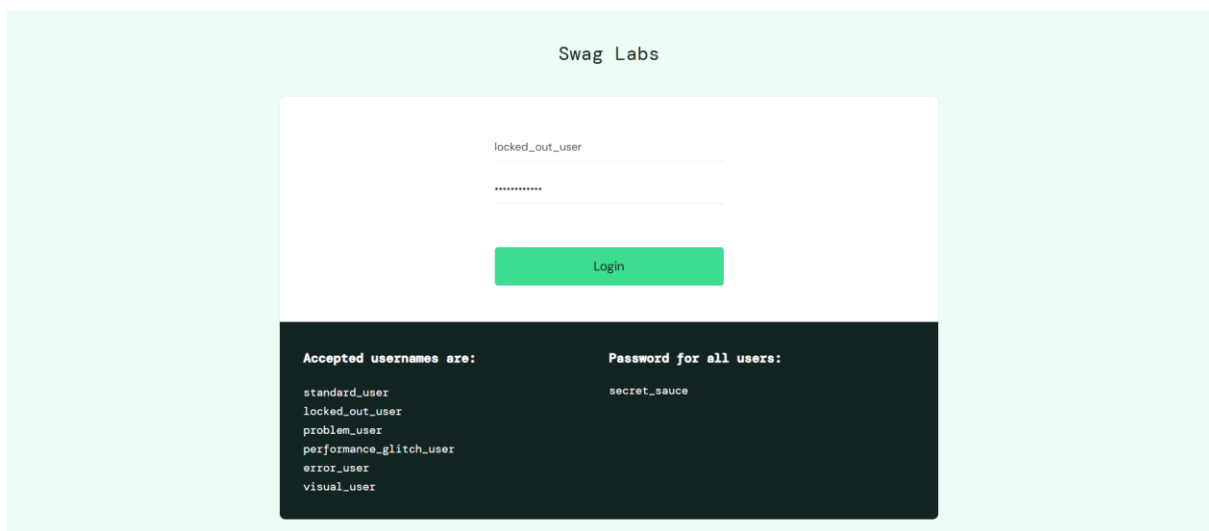
This screenshot shows the invalid username and password entered for the second test case.

Screenshot 5: Invalid Login Error Message



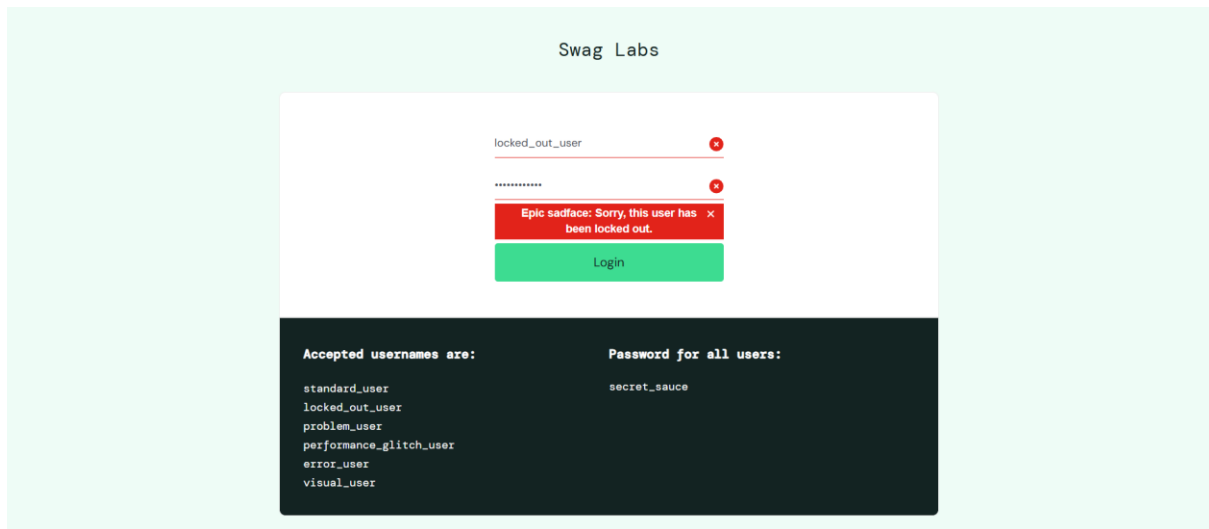
This screenshot displays the error message generated when the invalid credentials are submitted.

Screenshot 6: Locked-Out User Credentials Entered



This screenshot shows the credentials entered for the locked-out user test case.

Screenshot 7: Locked-Out User Error Message



This screenshot displays the error message shown when the locked-out user attempts to log in.

Screenshot 8: PyTest HTML Report

report.html

Report generated on 27-Sep-2026 at 17:00:36 by [pytest-html](#) v4.2.0

Environment

Python	3.10.0
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.6.0
Plugins	• anyio: 4.12.1 • html: 4.2.0 • metadata: 3.1.1

Summary

3 tests took 00:00:38.

(Un)check the boxes to filter the results.

☐

0 Failed

☒

3 Passed

☐

0 Skipped

☐

0 Expected failures

☐

0 Unexpected passes

☐

0 Errors

☐

0 Retuns

☐

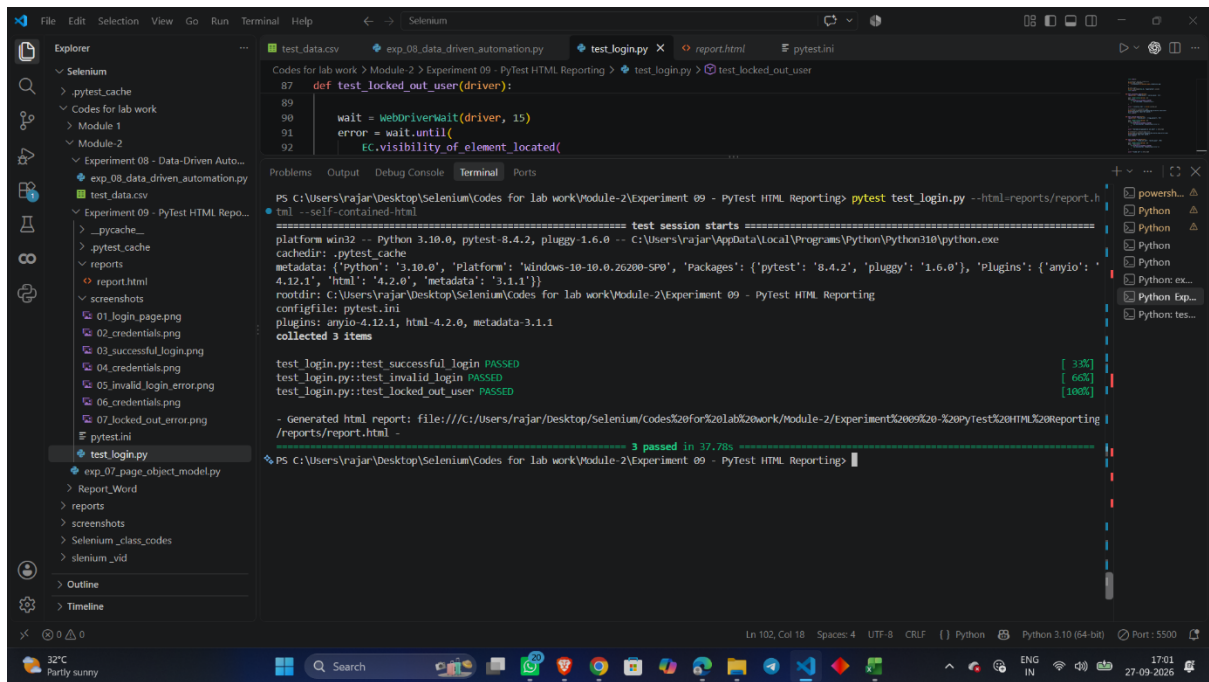
0 Retried

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	test_login.py::test_successful_login	00:00:13	
Passed	test_login.py::test_invalid_login	00:00:12	
Passed	test_login.py::test_locked_out_user	00:00:12	

This screenshot should show the HTML report in the browser, including the three test cases and their execution statuses.

Screenshot 9: Terminal Execution Output (Optional)



This screenshot should show the successful execution of all three test cases.

6. Result, Observation and Conclusion

Result

The Selenium automation script was successfully executed using the PyTest framework on the SauceDemo website.

The following three test cases were executed:

Test Case	Description	Status
Test Case 1	Successful login using valid credentials	PASSED
Test Case 2	Login using invalid credentials	PASSED
Test Case 3	Login using locked-out user credentials	PASSED

Execution Summary:

- Total test cases: 3
- Passed: 3
- Failed: 0
- Execution time: 13.76 seconds (initial run without the additional two-second delays)

The HTML report was generated successfully at `reports/report.html`.

Observation

1. PyTest successfully discovered and executed all three test cases.
2. Selenium WebDriver automated the login process in Chrome.
3. Explicit waits helped ensure that web elements were available before interaction.
4. Screenshots were captured at important stages of test execution.
5. The PyTest HTML plugin generated a report containing the test results.
6. The two-second delays made the browser interactions easier to observe during execution.

Conclusion

This experiment demonstrated how Selenium WebDriver can be integrated with PyTest to automate multiple web application test scenarios. PyTest fixtures simplified browser setup and teardown, while assertions verified the expected results of each test case.

The `pytest-html` plugin was used to generate an HTML report, providing a convenient way to review test execution results. Screenshot capture also provided visual evidence of the automation process.

The experiment helped demonstrate the use of PyTest for structured test execution, verification, and reporting in web automation.